

Billion-node 그래프의 부분그래프 매칭

Efficient Subgraph Matching on Billion Node Graphs

Zhao Sun, Hongzhi Wang, Haixun Wang, Bin Shao, Jianzhong Li (2012)

STwig 분해, 메모리 기반 탐색, 분산 join 최적화

부분그래프 매칭의 입력과 출력

라벨이 있는 질의 그래프의 모든 발생 위치를 큰 데이터 그래프에서 찾는다

01	데이터 그래프 G	정점, 간선, 정점 라벨로 구성
↓		
02	질의 그래프 q	필요한 라벨과 간선 관계를 지정
↓		
03	대응 f	질의 정점을 데이터 정점에 일대일 대응
↓		
04	매칭 결과	라벨과 질의 간선을 모두 보존한 부분그래프

이 논문은 하나의 매우 큰 그래프 안에서 부분그래프 매칭을 다룬다

왜 billion-node 규모에서 기존 방식이 압박을 받는가

질의마다 빠르게 찾기 위한 준비 작업이 그래프보다 더 커지거나 더 오래 걸릴 수 있다

- 그래프 탐색은 이웃으로의 임의 접근을 반복하므로 접근 지역성이 낮다
- 단일 장비 RAM에는 매우 큰 그래프를 모두 올리기 어렵다
- 2-hop 같은 구조 인덱스는 초선형 공간이나 구축 시간을 요구할 수 있다
- 간선 단위 분해 뒤의 multi-way join은 중간 결과를 크게 만들 수 있다

논문은 질의 실행 시간을 줄이는 구조 인덱스 자체가 규모의 병목이 될 수 있다고 본다

논문이 대비한 매칭 접근

Table 1의 분류는 방법의 절대적 서열이 아니라 billion-node 조건에서의 설계 부담을 보여준다

접근	준비 구조	병목	논문의 관점
무인덱스 탐색	별도 구조 없음	큰 탐색 공간	작은 그래프 실험 중심
간선 인덱스	edge index	다수 join	표현 가능한 질의도 제한
빈발 부분그래프	pattern index	발굴·저장 비용	질의 편향에 민감
도달성·이웃 인덱스	2-hop·반경 구조	초선형 구축·공간	web-scale에 부담
이 논문	라벨→ID index	탐색·필요 join	RAM 분산 탐색으로 보완

Trinity 위의 질의 경로

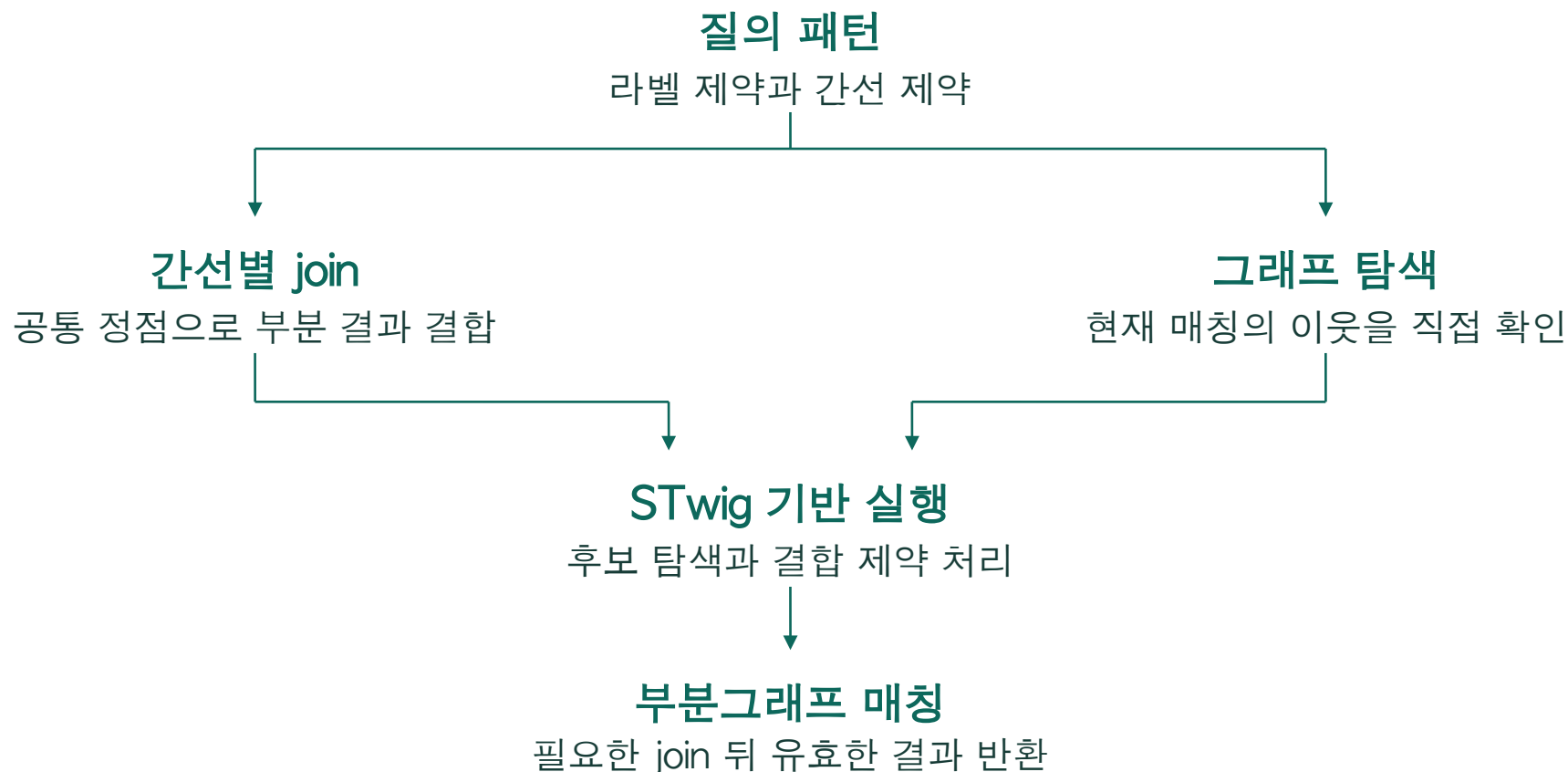
proxy가 계획을 만들고, 메모리 클러스터의 서버들이 탐색·결합을 나눠 수행한다



통합 주소 공간처럼 보이지만, 실제 데이터와 중간 결과는 여러 장비의 RAM에 나뉘어 있다

join과 탐색은 서로 대체되지 않는다

논문은 join을 완전히 없애기보다, 탐색으로 후보를 줄이고 필요한 join을 남긴다



STwig: 두 단계의 작은 질의 단위

STwig $q=(r,L)$ 는 루트 라벨 r 과 자식 라벨 집합 L 로 표현한다



STwig는 큰 구조 인덱스 대신 메모리에서 바로 검사할 수 있도록 의도적으로 작게 잡은 단위다

MatchSTwig가 하는 세 번의 접근

전체 그래프를 스캔하지 않고 라벨 후보에서 시작해 이웃 라벨을 확인한다



라벨→ID index는 이웃 구조를 미리 저장하는 graph-structure index와 다르다

강의용 축약 추적: a-b-c 패턴

원문 Fig. 1의 라벨을 사용해 STwig 후보와 전체 매칭의 차이를 분리해 본다

01	질의	a가 b와 c에 연결된 패턴
↓		
02	a 후보	라벨 index가 a1, a2를 찾는다
↓		
03	이웃 검사	각 a 후보에서 b·c 라벨 이웃을 찾는다
↓		
04	부분 결과	a-b-c 조합이 STwig 후보가 된다
↓		
05	전체 확인	남은 질의 간선과 정점 제약을 결합해 확인

STwig 후보 뒤에는 바인딩과 join이 필요하다

전체 매칭의 실행 흐름

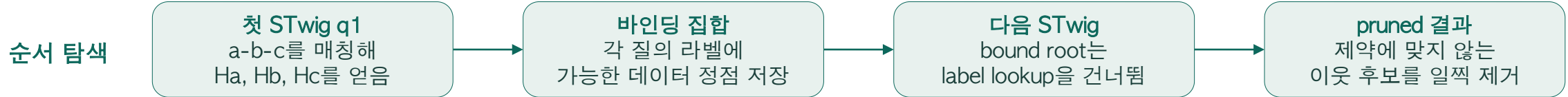
작은 STwig를 독립적으로 모두 join하지 않고, 앞 단계의 정보를 다음 탐색에 전달한다

01	분해와 순서	질의를 STwig cover로 나누고 실행 순서를 정함
↓		
02	순서 탐색	앞 STwig의 바인딩으로 다음 후보를 제한
↓		
03	부분 결과	각 STwig에서 pruning된 결과를 유지
↓		
04	join 순서 선택	작은 중간 결과부터 결합하도록 순서를 정함
↓		
05	최종 결과	각 서버의 결과를 합쳐 질의 응답을 만듦

탐색 단계는 join 전에 후보 공간을 줄이고, join 단계는 남은 전역 제약을 해결한다

바인딩 정보가 다음 탐색의 후보를 줄인다

앞 STwig가 매칭한 데이터 정점 집합을 보존해 다음 STwig의 루트나 자식을 제한한다



바인딩 집합은 답의 곱이 아니라 다음 검사의 후보 범위다

join은 남기되, 순서와 메모리 사용을 조절한다

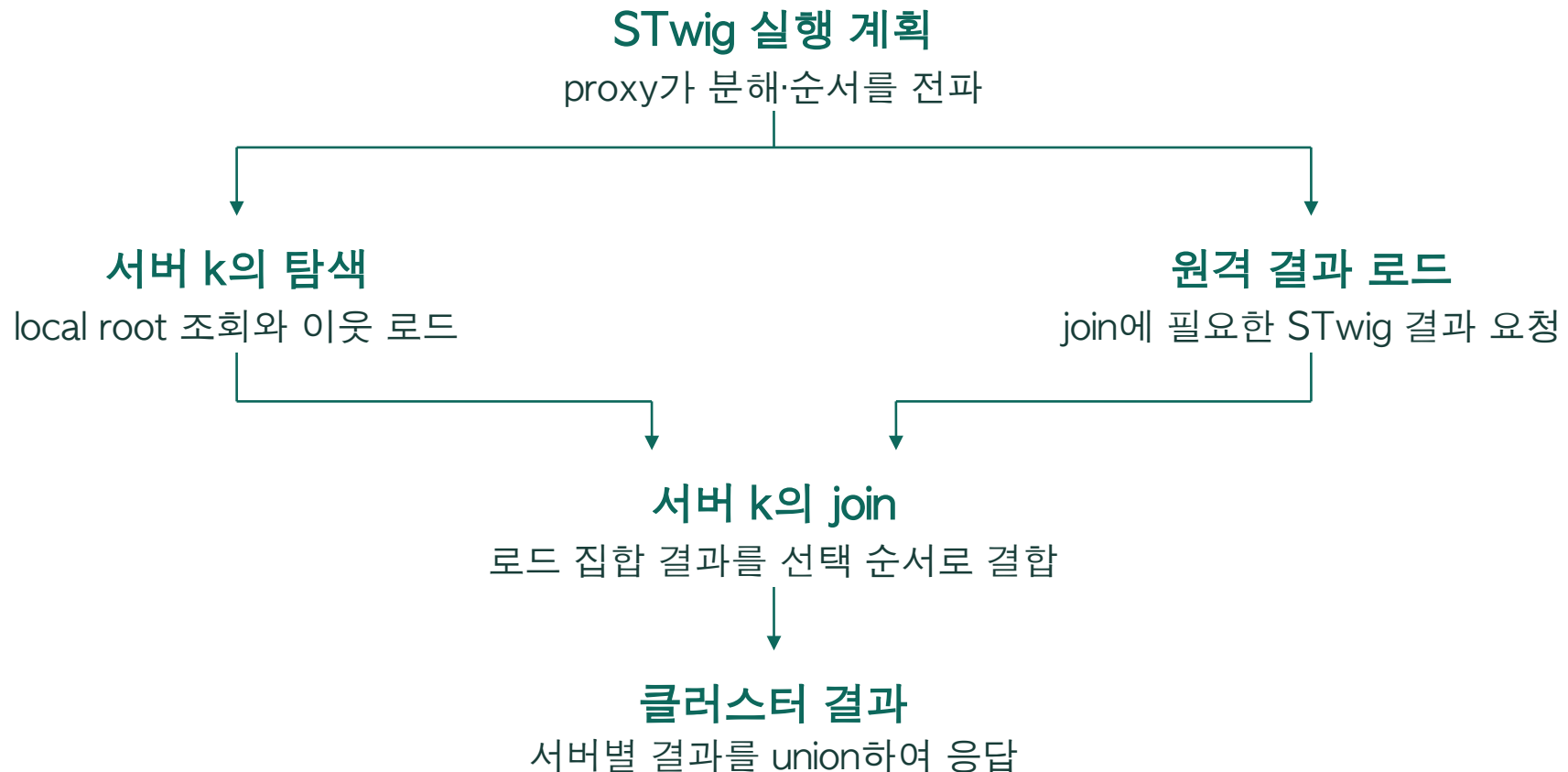
탐색 뒤에도 남는 STwig 결과는 join해야 하며, 중간 결과가 커질 수 있다

- | | | |
|----|---------------|---------------------------------|
| 01 | pruned 결과 수집 | 각 STwig의 탐색 결과 $G(q_i)$ 를 모음 |
| ↓ | | |
| 02 | join 비용 추정 | partial result 통계로 결합 순서를 선택 |
| ↓ | | |
| 03 | block 분할 | 각 STwig 결과 일부만 한 라운드에 참여 |
| ↓ | | |
| 04 | pipeline join | 전체 multi-way join 전에도 부분 결과를 생성 |
| ↓ | | |
| 05 | 메모리 제어 | 사용 가능한 메모리로 block 크기를 조절 |

pipeline join은 중간 결과의 동시 메모리 점유를 제한한다

분산 실행에서 탐색과 결합의 역할

분해·순서는 proxy가 정하고, 각 서버는 자신의 partition에서 탐색한 뒤 필요한 결과를 교환한다



헤드 STwig가 중복 없는 결과의 기준점이 된다

각 서버가 헤드 STwig는 로컬 결과만 사용하도록 고정한다

01	헤드 qs 선택	분해된 STwig 하나를 기준으로 정함
↓		
02	로컬 헤드 결과	각 서버 k에서 $G_k(qs)$ 만 사용
↓		
03	서버 partition	데이터 partition이 서로 겹치지 않음
↓		
04	서버별 최종 결과	서로 다른 헤드 매칭을 anchor로 삼음
↓		
05	union	원문 모델에서는 별도 deduplication 없이 합침

중복 회피는 local head anchor와 disjoint partition에 따른다

STwig cover 수를 줄이는 이유

STwig가 많아질수록 join과 원격 중간 결과 교환이 늘어날 수 있다

- | | | |
|----|-------------|------------------------------|
| 01 | 질의 간선 | 모든 질의 간선을 한 번씩 덮어야 함 |
| ↓ | | |
| 02 | STwig cover | 각 간선을 정확히 한 STwig에 배정 |
| ↓ | | |
| 03 | 최소 cover | STwig 수와 잠재 join 수를 줄이려 함 |
| ↓ | | |
| 04 | 난해성 | minimum vertex cover와 다항식 동치 |
| ↓ | | |
| 05 | 근사 | 논문은 2-근사 cover를 사용 |

2-근사는 STwig cover의 크기에 대한 보장이지, 질의 전체 실행 시간의 2배 보장이 아니다

Algorithm 2는 cover와 실행 순서를 함께 만든다

이미 선택된 간선에 연결되고 selectivity가 높은 간선을 우선해 다음 루트를 가능한 한 bound 상태로 만든다



높은 f-value는 라벨이 드물고 연결 요구가 많은 쪽을 우선하려는 휴리스틱이다

클러스터 그래프로 원격 로드 범위를 줄인다

모든 서버에서 모든 STwig 결과를 가져오지 않도록, 질의와 데이터 분포로 가능한 서버만 정한다

01	관련 그래프 G_q	질의 간선과 맞지 않는 데이터 간선을 제외한 개념적 부분
↓		
02	클러스터 그래프 C	서버가 정점, 관련 교차 partition 연결이 간선
↓		
03	거리 DC	서버 k 와 j 사이의 최단 거리
↓		
04	질의 거리 d	헤드와 대상 STwig 루트 사이의 최단 거리
↓		
05	로드 집합 $F_{k,t}$	$DC(k,j)$ 가 d 이하인 서버 j 만 요청

load set은 서버별 라벨 쌍 기록을 lookup해 구성한다

헤드 STwig 선택은 통신 반경을 줄이는 문제

헤드의 루트가 다른 STwig 루트에서 멀수록 더 많은 서버가 로드 집합에 포함될 수 있다

01	STwig 후보 q_i	각 STwig의 루트 r_i 를 확인
↓		
02	질의 최단거리	질의 그래프에서 r_i 와 r_j 의 거리를 계산
↓		
03	최대 반경 $d(i)$	후보 i 에서 다른 모든 루트까지의 최대 거리
↓		
04	최소 후보 선택	$d(i)$ 가 가장 작은 STwig를 head로 선택
↓		
05	통신 비용	원문 Eq. 2의 load-set 크기 합을 줄이는 방향

head 선택은 질의 내 거리로 통신 반경을 줄인다

실험 조건을 결과와 함께 읽기

논문의 수치는 2012년 Trinity 구현과 특정 데이터·쿼리·클러스터 조건에서 얻었다

항목	실제 데이터	합성 데이터	해석 경계
클러스터	8대, 각 32 GB RAM	12대, 각 48 GB RAM	서로 다른 cluster
그래프	US Patents, WordNet	R-MAT	동일 workload 아님
질의	데이터별 100개 평균	DFS-random 생성	pipeline은 1,024 match 뒤 종료
기본값	라벨 집합 사용	64M nodes, degree 64	조건을 고정해 변수별 비교
구현	C# / .NET 4	C# / .NET 4	현대 하드웨어 일반화 불가

실제 데이터에서 보고된 경향

Fig. 8-9는 query 생성 방식과 서버 수가 runtime에 미치는 영향을 별도로 보여준다

관찰	조건	원문의 해석
DFS 질의 크기	3-10개 노드, Patents·WordNet	STwig·join 증가와 pruning 효과가 함께 나타남
random 질의 크기	5-15개 노드, 간선 수 2N	대체로 query 노드 수에 거의 선형
간선 수	10-20개 간선 실험	평균 STwig·join 수가 크게 늘지 않아 영향이 작았음
서버 수	동일 질의를 1-8대에서 실행	runtime은 감소했지만 network·sync로 sub-linear speedup

성능은 질의 형태와 라벨 밀도, 중간 결과를 함께 봐야 한다

합성 그래프 결과의 조건부 읽기

Fig. 10은 한 변수를 바꾸고 나머지를 고정한 synthetic R-MAT 실험이다

변수	고정 또는 범위	보고된 관찰
그래프 크기	1M-4,096M nodes, degree 16 고정	응답 시간 약 400-1,800 ms, 명확한 비례 관계 없음
노드 수(밀도 고정)	node count 변화, density 고정	평균 degree 증가와 함께 runtime 증가
평균 degree	5-65 범위	이웃 접근과 중간 결과가 늘어 runtime 증가
라벨 밀도	10^{-5} - 10^{-1}	라벨 후보가 줄수록 runtime 감소

논문의 ‘sub-linear’ 관찰은 제시된 R-MAT 파라미터와 질의 workload 안에서 읽어야 한다

이 논문의 설계상 차별점

차별점은 하나의 기법이 아니라, 가벼운 index와 탐색·join·통신 계획의 결합이다

설계 요소	무엇을 피하는가	대신 수행하는 일
구조 인덱스 미사용	초선형 index 공간·구축	라벨→ID lookup과 이웃 탐색
STwig 분해	큰 pattern index	작은 두 단계 단위의 후보 생성
바인딩 기반 순서	불필요한 root lookup	앞 결과로 다음 후보를 제한
load set	전 서버 결과 broadcast	질의·cluster 거리로 원격 요청 제한
pipeline join	중간 결과의 한꺼번에 보관	block 단위로 join 진행

해석할 때 남겨야 할 한계

이 논문이 해결한 병목과 남아 있는 병목을 구분해야 한다

- cycle과 전역 제약은 join 또는 동등한 결합 검사를 요구할 수 있다
- 드문 라벨과 강한 바인딩이 없으면 STwig 후보와 중간 결과가 커질 수 있다
- 2-근사 보장은 STwig cover 수에 관한 것이며 end-to-end latency 보장이 아니다
- 결과는 Trinity 기반 2012년 hardware-network-workload 조건의 실험값이다
- 원문도 더 큰 graph, throughput, transmitted data 측정을 future work로 남겼다

빠른 메모리 이웃 접근과 작은 라벨 index가 이 설계의 전제다

한 장으로 다시 보는 STwig 매칭

거대 graph를 미리 세밀하게 index하지 않고, 질의 때 작은 단위로 탐색·제한·결합한다

01	질의 입력	라벨과 간선으로 부분그래프 패턴을 표현
↓		
02	STwig 계획	cover·순서·head를 정해 탐색 범위를 설계
↓		
03	분산 탐색	각 RAM partition에서 후보와 binding을 생성
↓		
04	선별된 join	load set과 join order로 남은 제약을 결합
↓		
05	조건부 성능	graph·질의·cluster 조건에서 결과를 해석

핵심은 index와 질의 시점 제약 처리의 역할을 다시 나누는 것이다